

PRAKTIKUM TEKNOLOGI IOT II

LAPORAN PRAKTIK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum Mata Kuliah
Teknologi IOT II



DISUSUN OLEH:

NAMA : Tengku Geisya Gustiana Syah

NPM : 25782028

KELAS :3A

PRODI : TEKNOLOGI REKAYASA INTERNET

Dosen :

Dr. Ir. Septafiansyah Dwi Putra, S. T., M.T.

Agus Ambarwari, S.Pd.,M.kom,

TEKNOLOGI REKAYASA INTERNET

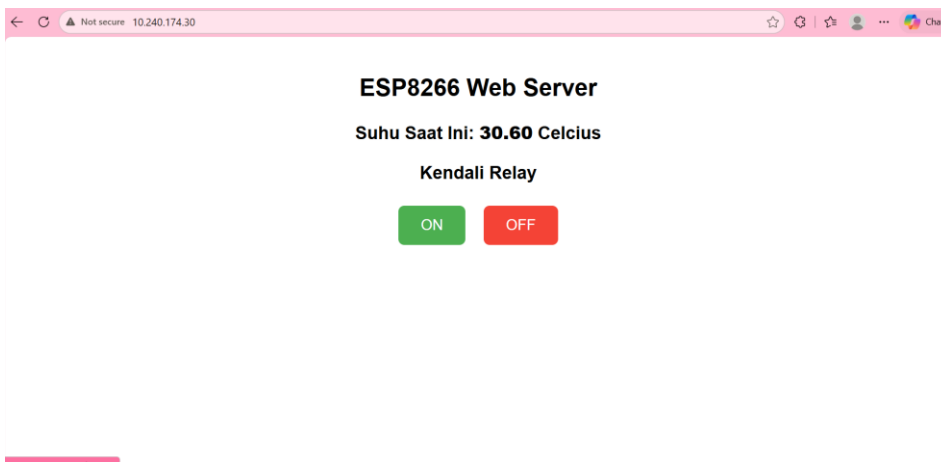
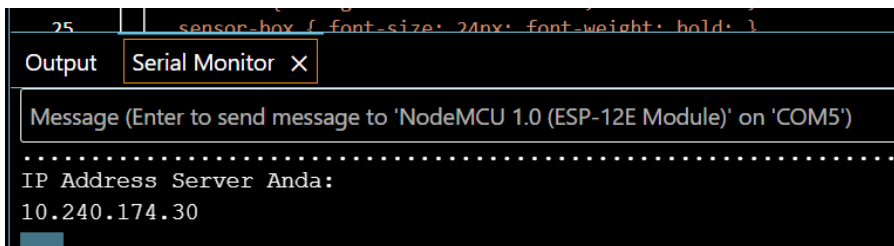
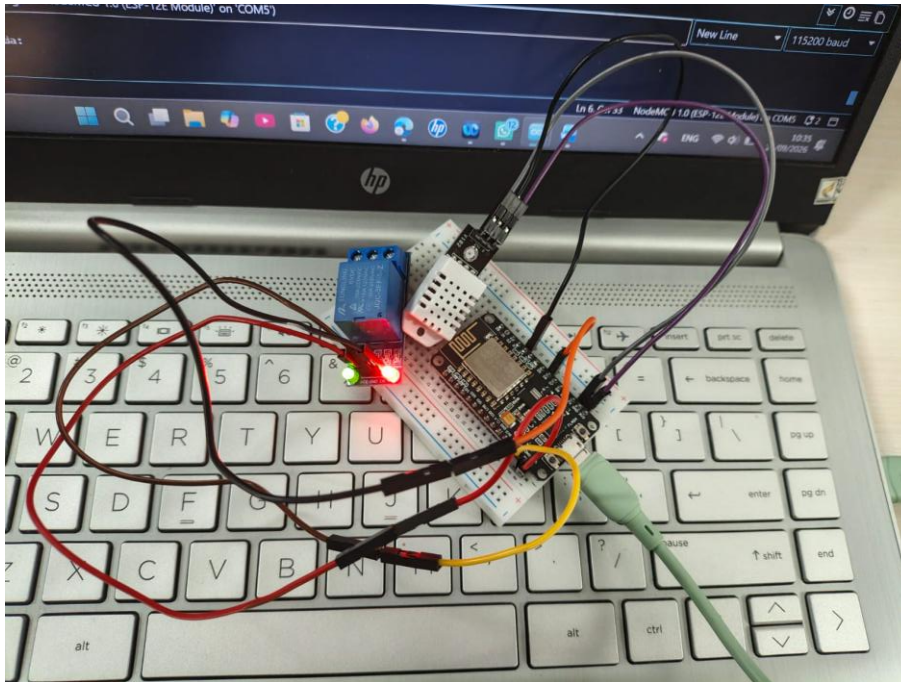
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

3.5 Praktikum

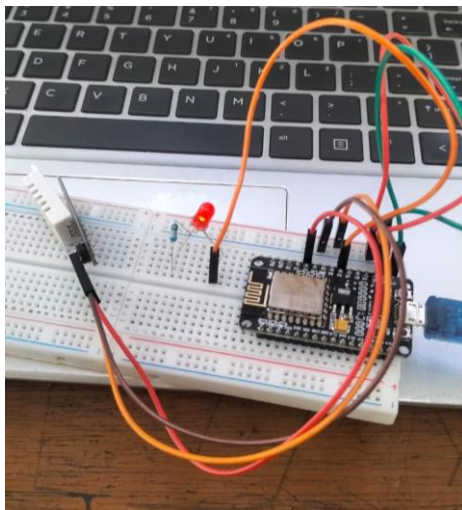
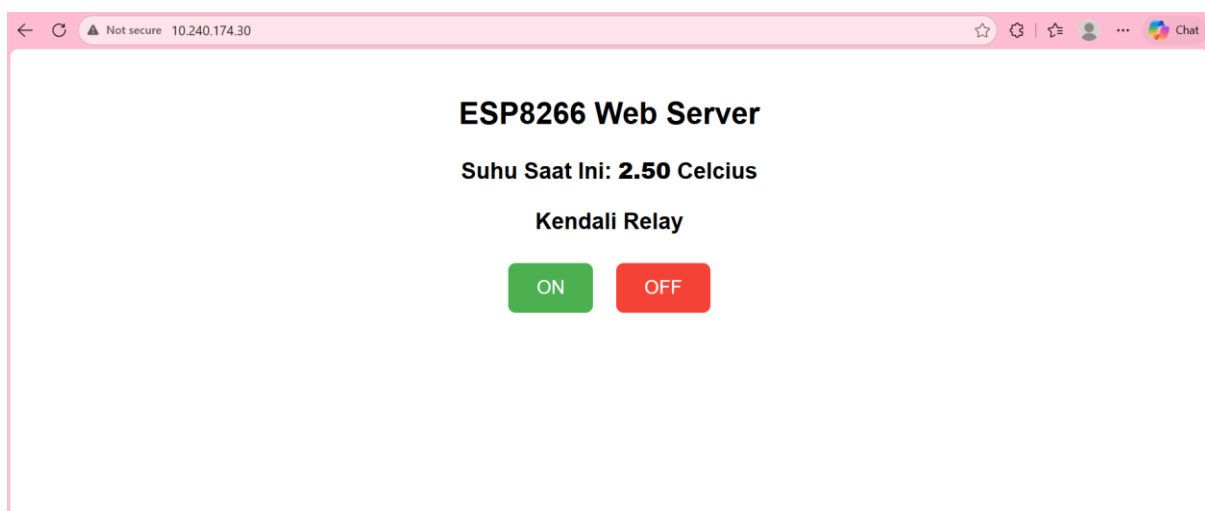
3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)



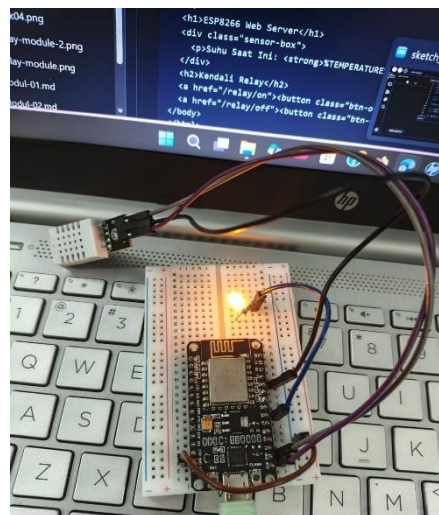
Pada praktikum ini, NodeMCU digunakan sebagai **web server**. Jadi NodeMCU disambungkan ke Wi-Fi, lalu kita bisa membuka halaman web dari HP atau laptop untuk melihat suhu dan mengontrol relay. Sensor **DHT22** dipasang ke pin D4 untuk membaca suhu. Relay dipasang ke pin D6. Setelah program di-upload, NodeMCU akan terhubung ke Wi-Fi dan menampilkan **alamat IP** di Serial Monitor.

Alamat IP tersebut kemudian dibuka melalui browser. Di halaman web akan muncul **nilai suhu** dari DHT22 dan tombol **ON/OFF** untuk mengatur relay. Jadi sederhananya, praktikum ini mencoba membuat **NodeMCU bisa dipantau dan dikontrol lewat browser menggunakan Wi-Fi**, bukan cuma membaca sensor lewat Serial Monitor.

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)



Client



Server

Pada praktikum ini, NodeMCU digunakan sebagai **HTTP Client** yang berfungsi membaca kondisi cahaya menggunakan sensor LDR. LDR dihubungkan ke pin A0 sehingga nilai intensitas cahaya dapat dibaca oleh NodeMCU.

Jika nilai yang terbaca dari LDR **kurang dari 300**, kondisi dianggap gelap. NodeMCU kemudian mengirimkan request **HTTP GET** melalui jaringan Wi-Fi ke NodeMCU lain yang sebelumnya sudah

dijadikan **Web Server** pada praktikum 3.5.1. Request tersebut digunakan untuk menjalankan perintah /relay/on, sehingga relay pada NodeMCU Web Server dapat menyala. Dengan praktikum ini, dapat dilihat bahwa dua NodeMCU dapat saling berkomunikasi melalui jaringan Wi-Fi. NodeMCU pertama bertugas menerima perintah dan mengendalikan relay, sedangkan NodeMCU kedua membaca sensor LDR dan mengirimkan perintah melalui HTTP.

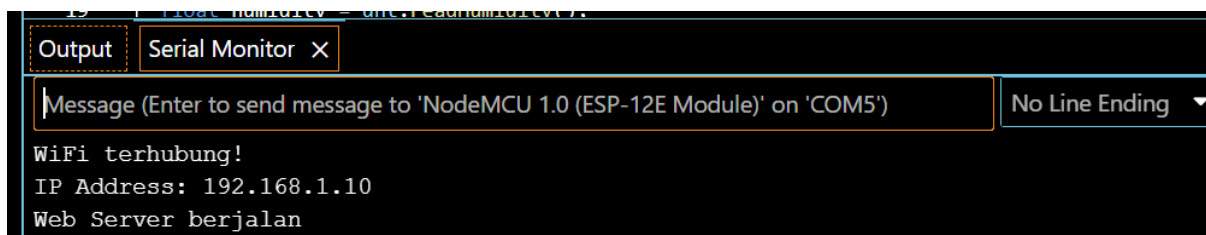
3.6 Latihan

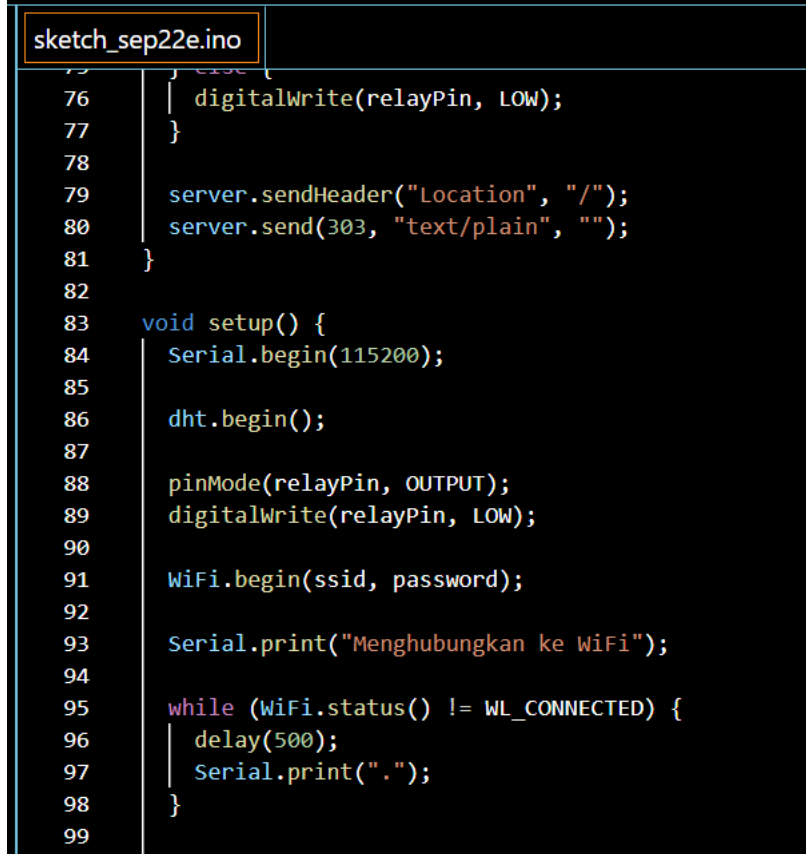
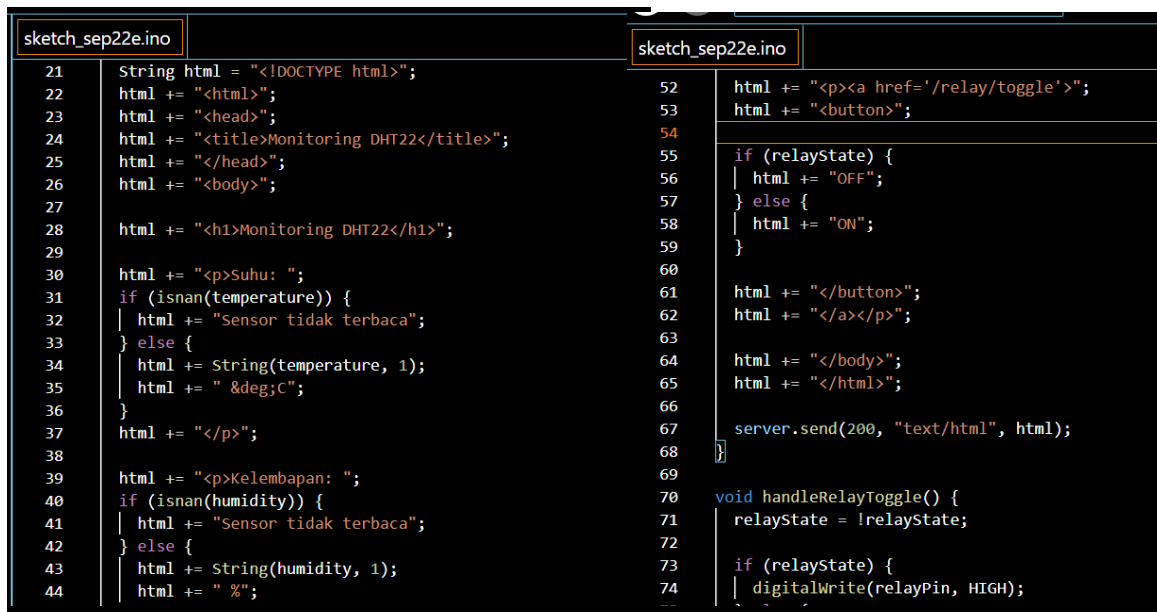
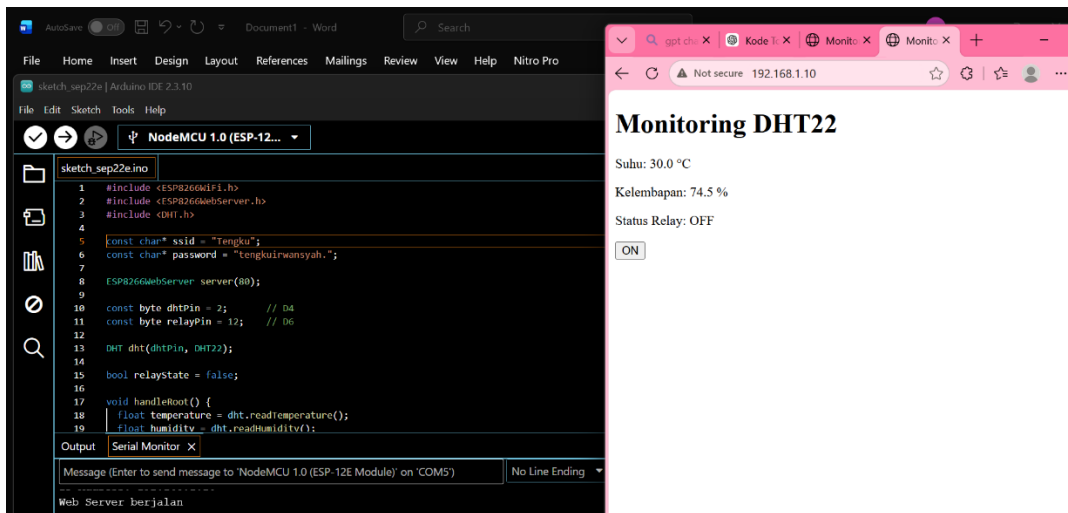
1. **(Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

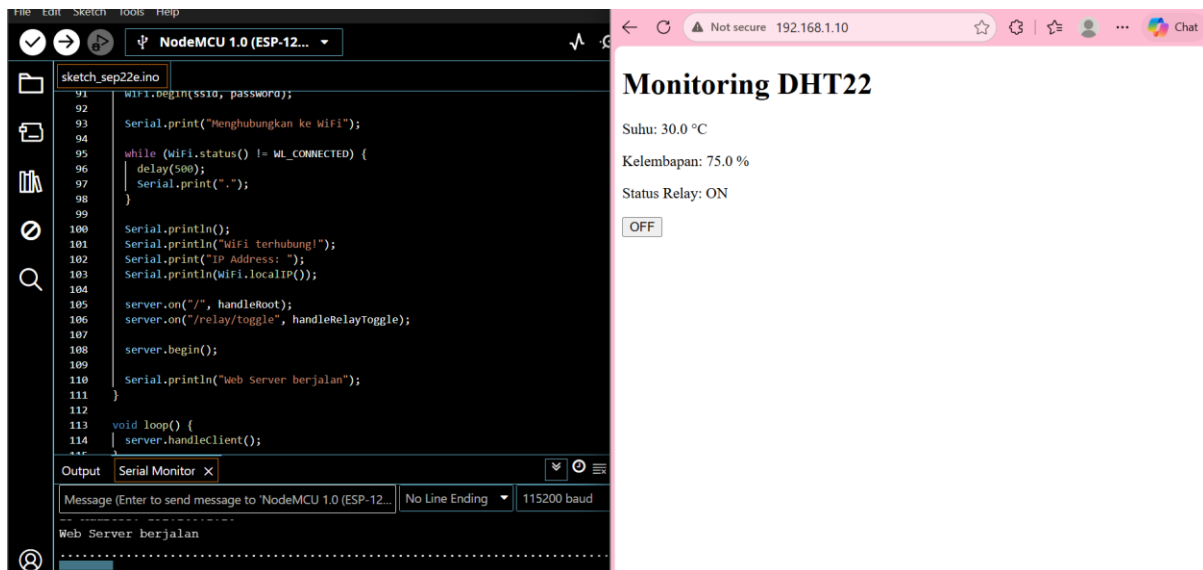
Jawab: Mode Station lebih sering digunakan karena NodeMCU bertindak sebagai perangkat yang terhubung ke Wi-Fi yang sudah tersedia. Dengan begitu, NodeMCU bisa berkomunikasi dengan perangkat lain yang berada di jaringan yang sama. Sedangkan pada mode AP, NodeMCU membuat jaringan Wi-Fi sendiri sehingga pengguna harus terhubung langsung ke jaringan yang dibuat oleh NodeMCU.

2. **(Modifikasi Antarmuka & Telemetry):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

- **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.
- **Tombol Dinamis Tunggal (Toggle Button):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi LED saat ini:







3. **(Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

Jawab: jika **Web Server sedang offline**, maka NodeMCU Client tidak dapat terhubung ke server yang dituju. Akibatnya, request **HTTP GET** yang dikirim oleh Client tidak mendapatkan respons dari server dan perintah untuk mengaktifkan relay tidak dapat dijalankan.

Pada *Serial Monitor*, kode respons HTTP tidak akan menunjukkan **200**, karena server tidak memberikan respons. Hal ini menunjukkan bahwa komunikasi antara NodeMCU Client dan Web Server tidak berhasil

4. Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

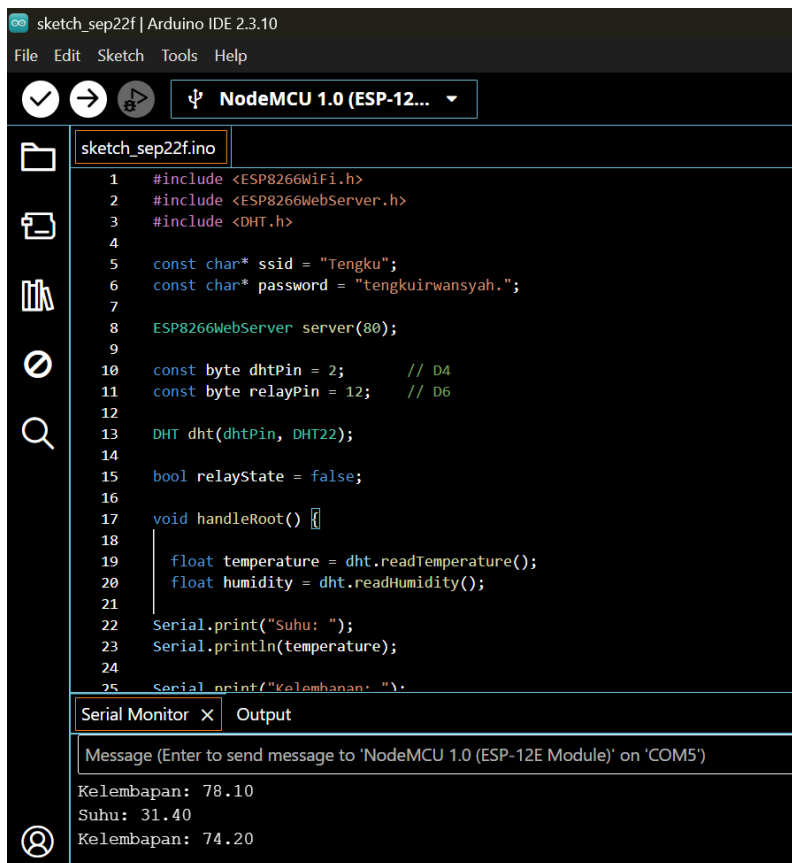
Jawab: Pada latihan ini, kondisi pada if diubah dari pembacaan sensor LDR menjadi pembacaan suhu dari DHT. Client hanya akan mengirim **GET Request** jika suhu yang terbaca **lebih dari 35°C**.

Baris algoritmanya:

```
if (temperature > 35) {  
    // kirim GET Request  
}
```

Jadi, jika suhu **> 35°C**, Client mengirim GET Request. Jika suhu **≤ 35°C**, Client tidak mengirim request.

3.7 Tugas

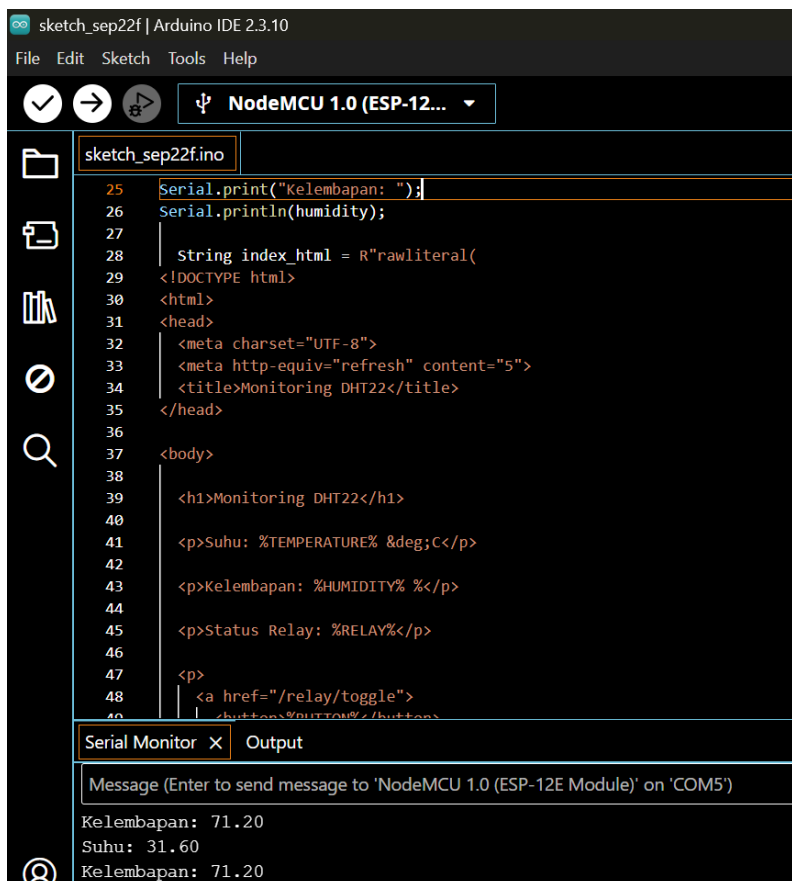


```
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Tengku";
6  const char* password = "tengkuirwansyah.";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;    // D4
11 const byte relayPin = 12; // D6
12
13 DHT dht(dhtPin, DHT22);
14
15 bool relayState = false;
16
17 void handleRoot() {
18     float temperature = dht.readTemperature();
19     float humidity = dht.readHumidity();
20
21     Serial.print("Suhu: ");
22     Serial.println(temperature);
23
24     Serial.print("Kelembapan: ");
25     Serial.println(humidity);
26 }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan: 78.10
Suhu: 31.40
Kelembapan: 74.20



```
25 Serial.print("Kelembapan: ");
26 Serial.println(humidity);
27
28 String index_html = R"rawliteral(
29 <!DOCTYPE html>
30 <html>
31 <head>
32   <meta charset="UTF-8">
33   <meta http-equiv="refresh" content="5">
34   <title>Monitoring DHT22</title>
35 </head>
36
37 <body>
38
39   <h1>Monitoring DHT22</h1>
40
41   <p>Suhu: %TEMPERATURE% &deg;C</p>
42
43   <p>Kelembapan: %HUMIDITY% %</p>
44
45   <p>Status Relay: %RELAY%</p>
46
47   <p>
48     <a href="/relay/toggle">
49       <button>%BUTTON%</button>
50     </a>
51   </p>
52 </body>
53 </html>
54 <rawliteral>
55
56 void handleRoot() {
57     float temperature = dht.readTemperature();
58     float humidity = dht.readHumidity();
59
60     Serial.print("Suhu: ");
61     Serial.println(temperature);
62
63     Serial.print("Kelembapan: ");
64     Serial.println(humidity);
65
66     server.send(200, "text/html", index_html);
67 }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan: 71.20
Suhu: 31.60
Kelembapan: 71.20

sketch_sep22f | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

sketch_sep22f.ino

```
52 |  
53 |  
54 |</body>  
55 |</html>  
56 |)rawliteral";  
57 |  
58 | if (isnan(temperature)) {  
59 | | index_html.replace("%TEMPERATURE%", "Sensor tidak terbaca");  
60 | } else {  
61 | | index_html.replace("%TEMPERATURE%", String(temperature, 1));  
62 | }  
63 |  
64 | if (isnan(humidity)) {  
65 | | index_html.replace("%HUMIDITY%", "Sensor tidak terbaca");  
66 | } else {  
67 | | index_html.replace("%HUMIDITY%", String(humidity, 1));  
68 | }  
69 |  
70 | if (relayState) {  
71 | | index_html.replace("%RELAY%", "ON");  
72 | | index_html.replace("%BUTTON%", "OFF");  
73 | } else {  
74 | | index_html.replace("%RELAY%", "OFF");  
75 | | index_html.replace("%BUTTON%", "ON");  
76 | }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan: 71.60
Suhu: 31.50
Kelembapan: 71.70

sketch_sep22f | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

sketch_sep22f.ino

```
53 |  
54 |</body>  
55 |</html>  
56 |)rawliteral";  
57 |  
58 | if (isnan(temperature)) {  
59 | | index_html.replace("%TEMPERATURE%", "Sensor tidak terbaca");  
60 | } else {  
61 | | index_html.replace("%TEMPERATURE%", String(temperature, 1));  
62 | }  
63 |  
64 | if (isnan(humidity)) {  
65 | | index_html.replace("%HUMIDITY%", "Sensor tidak terbaca");  
66 | } else {  
67 | | index_html.replace("%HUMIDITY%", String(humidity, 1));  
68 | }  
69 |  
70 | if (relayState) {  
71 | | index_html.replace("%RELAY%", "ON");  
72 | | index_html.replace("%BUTTON%", "OFF");  
73 | } else {  
74 | | index_html.replace("%RELAY%", "OFF");  
75 | | index_html.replace("%BUTTON%", "ON");  
76 | }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan: 72.40
Suhu: 31.30
Kelembapan: 72.50

sketch_sep22f | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

sketch_sep22f.ino

```
76
77     server.send(200, "text/html", index_html);
78 }
79
80 void handleRelayToggle() {
81
82     relayState = !relayState;
83
84     if (relayState) {
85         digitalWrite(relayPin, HIGH);
86     } else {
87         digitalWrite(relayPin, LOW);
88     }
89
90     server.sendHeader("Location", "/");
91     server.send(303, "text/plain", "");
92 }
93
94 void setup() {
95
96     Serial.begin(115200);
97     delay(1000);
98
99     dht.begin();
100 }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan: 72.80
Suhu: 31.20
Kelembapan: 72.70

sketch_sep22f | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

sketch_sep22f.ino

```
100
101     pinMode(relayPin, OUTPUT);
102     digitalWrite(relayPin, LOW);
103
104     Serial.println();
105     Serial.println("Menghubungkan ke WiFi...");
106
107     WiFi.begin(ssid, password);
108
109     int attempts = 0;
110
111     while (WiFi.status() != WL_CONNECTED && attempts < 30) {
112         delay(500);
113         Serial.print(".");
114         attempts++;
115     }
116
117     Serial.println();
118
119     if (WiFi.status() == WL_CONNECTED) {
120
121         Serial.println("WiFi terhubung!");
122
123         Serial.print("IP Address: ");
124         Serial.println(WiFi.localIP());
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

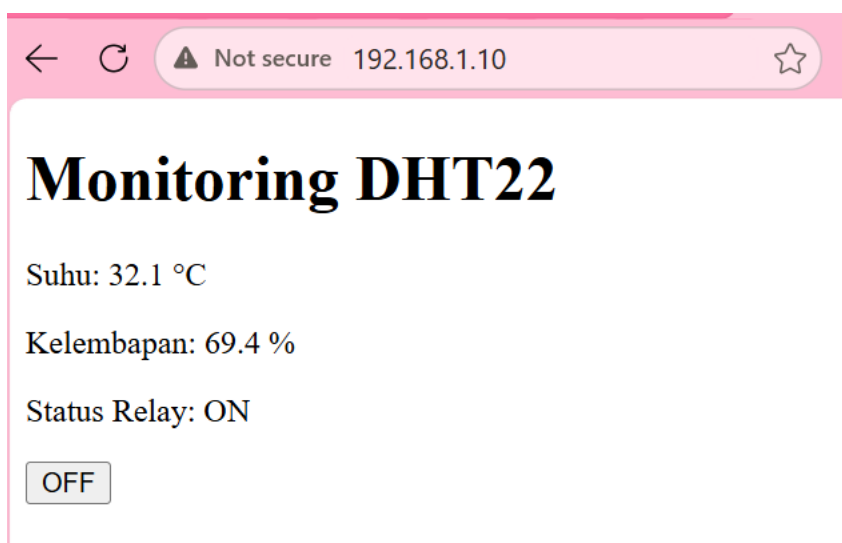
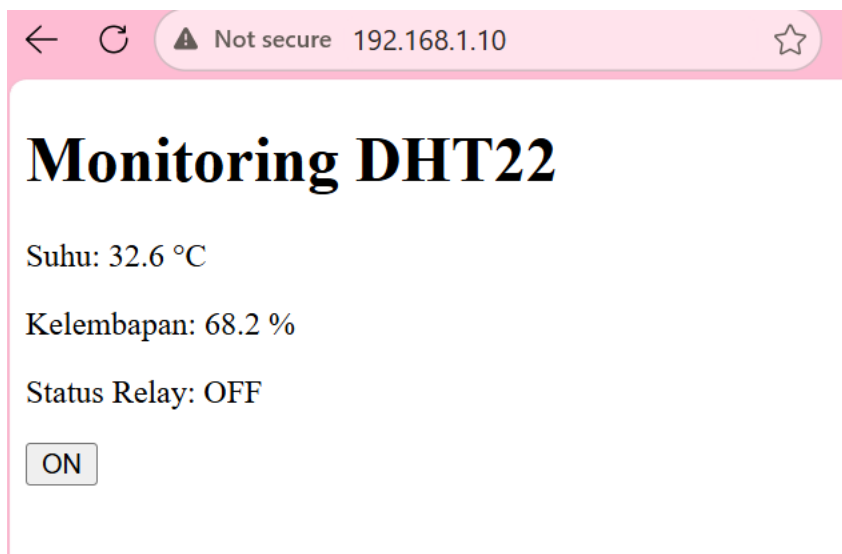
Kelembapan: 72.70
Suhu: 31.10
Kelembapan: 72.60

```
124 Serial.println(WiFi.localIP());
125
126 server.on("/", handleRoot);
127 server.on("/relay/toggle", handleRelayToggle);
128
129 server.begin();
130
131 Serial.println("Web Server berjalan");
132
133 } else {
134   Serial.println("Gagal terhubung ke WiFi!");
135 }
136 }
137 }
138
139 void loop() {
140   server.handleClient();
141 }
142
143 }
```

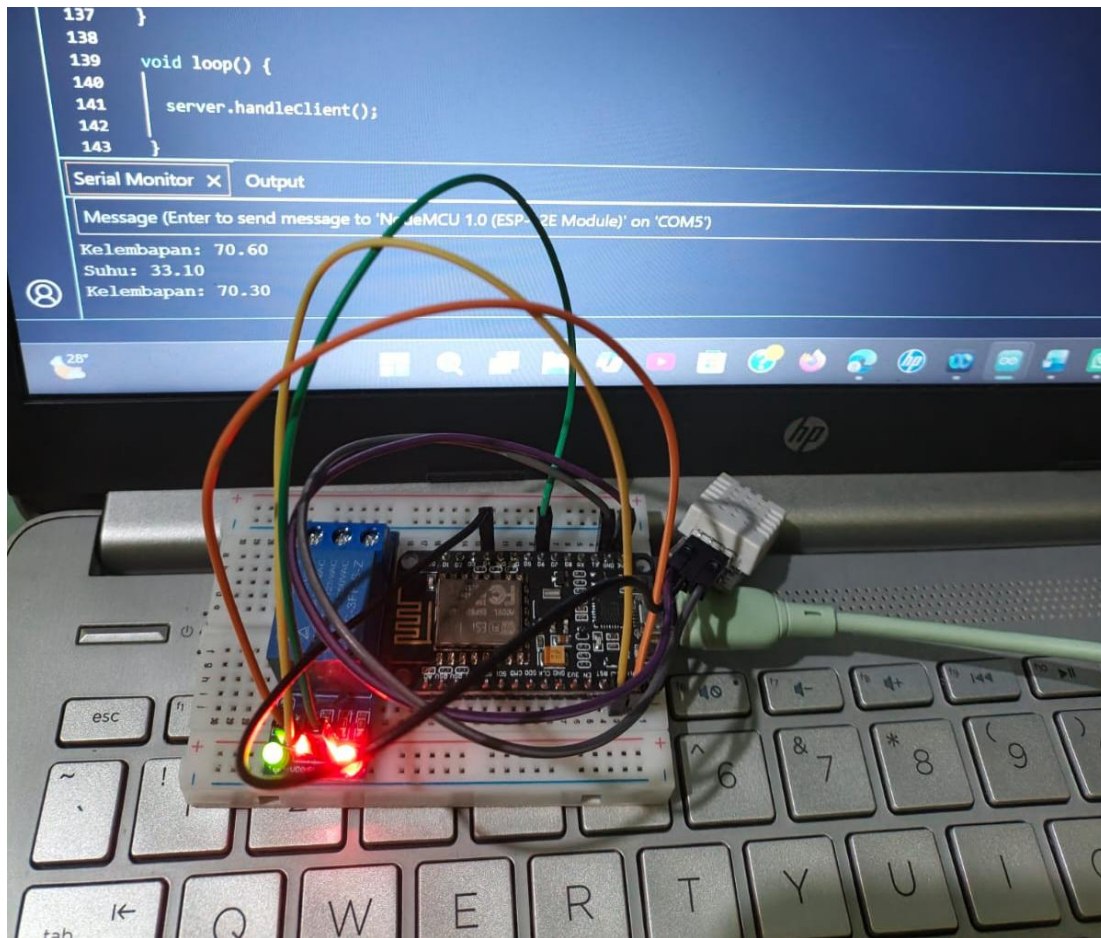
Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan: 72.90
Suhu: 31.10
Kelembapan: 73.00



Hasil pengujian Web Server dengan fitur Meta Refresh 5 detik. Halaman web menampilkan data suhu dan kelembapan dari sensor DHT22 serta status relay. Halaman diperbarui secara otomatis setiap 5 detik sehingga data dapat berubah tanpa melakukan refresh secara manual.



Pada Tugas 3.7 dilakukan modifikasi pada program Web Server dari Latihan 2 dengan menambahkan fitur **Meta Refresh** pada halaman web. Meta Refresh diatur dengan interval 5 detik menggunakan kode berikut:

```
<meta http-equiv="refresh" content="5">
```

Berdasarkan hasil pengujian, halaman Web Server berhasil menampilkan data **suhu dan kelembapan dari sensor DHT22** serta status relay. Halaman juga melakukan refresh secara otomatis setiap 5 detik tanpa perlu melakukan refresh secara manual. Dengan demikian, perubahan nilai suhu dan kelembapan dapat ditampilkan secara berkala pada halaman web.

Analisis

Penggunaan Meta Refresh membuat proses monitoring menjadi lebih mudah karena halaman web dapat diperbarui secara otomatis setiap 5 detik. Data suhu dan kelembapan dari DHT22 akan diperbarui ketika halaman melakukan reload.

Namun, metode ini memiliki beberapa kekurangan. Karena seluruh halaman dimuat ulang setiap 5 detik, tampilan dapat mengalami **flickering** atau berkedip. Selain itu, proses reload secara terus-menerus menyebabkan halaman dikirim kembali sehingga penggunaan bandwidth menjadi lebih besar. Penggunaan tombol ON/OFF relay juga dapat terganggu apabila pengguna menekan tombol ketika halaman sedang melakukan refresh.

Dari hasil pengujian, Meta Refresh dapat digunakan untuk memperbarui data monitoring secara sederhana dan berkala. Namun, metode ini kurang efisien jika digunakan pada sistem yang membutuhkan pembaruan data secara real-time dan interaksi pengguna yang lebih lancar.